

Digital Image Processing



Lecture 10

Image Segmentation - II

Dr. Muhammad Sajjad
R.A: Asad Ullah

Overview

Recap of Traditional Methods 1

Evaluation Metrics 2

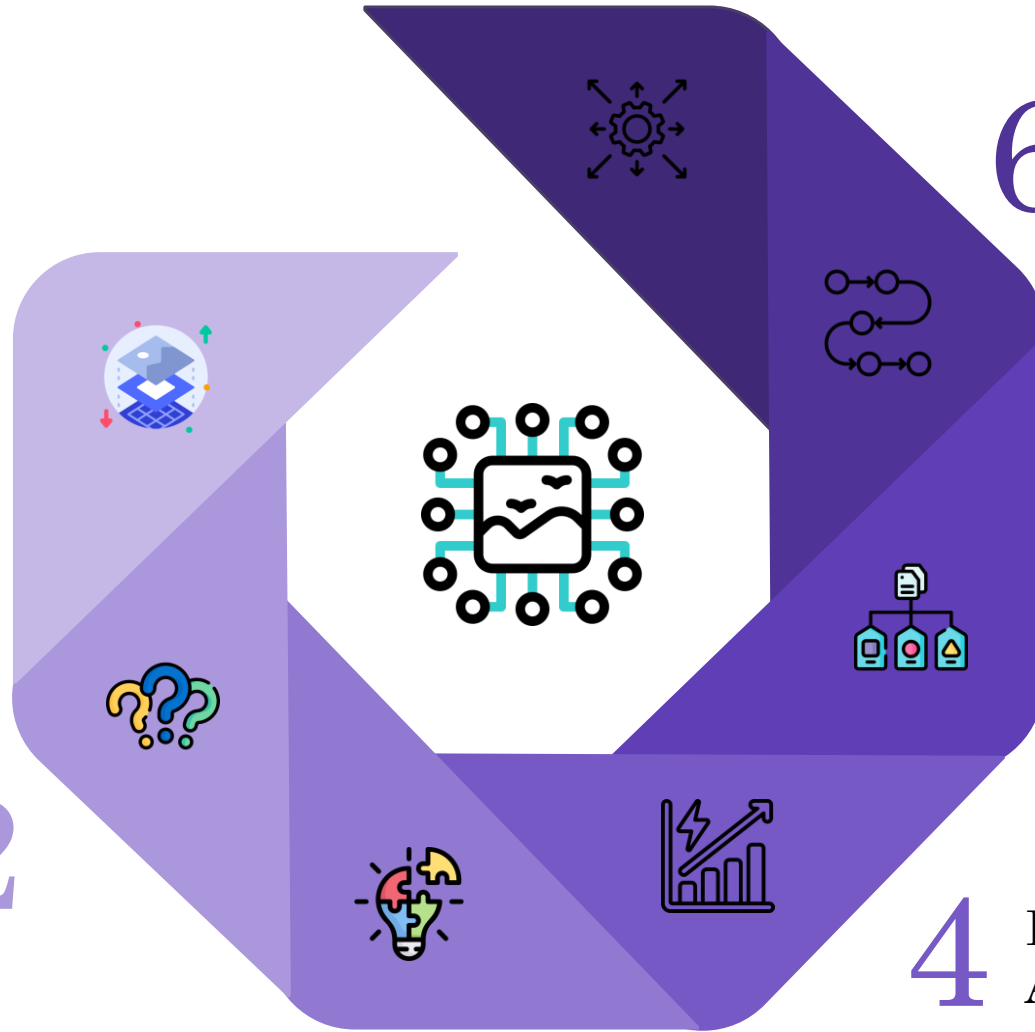
The Deep Learning Shift 3

7 Instance & Real-Time Segmentation

6 Advanced Semantic Models

5 Encoder-Decoder Models

4 Foundational Architectures (FCN)



PART 4: DEEP LEARNING ERA

The Deep Learning Revolution - Why It Changed Everything

The Paradigm Shift

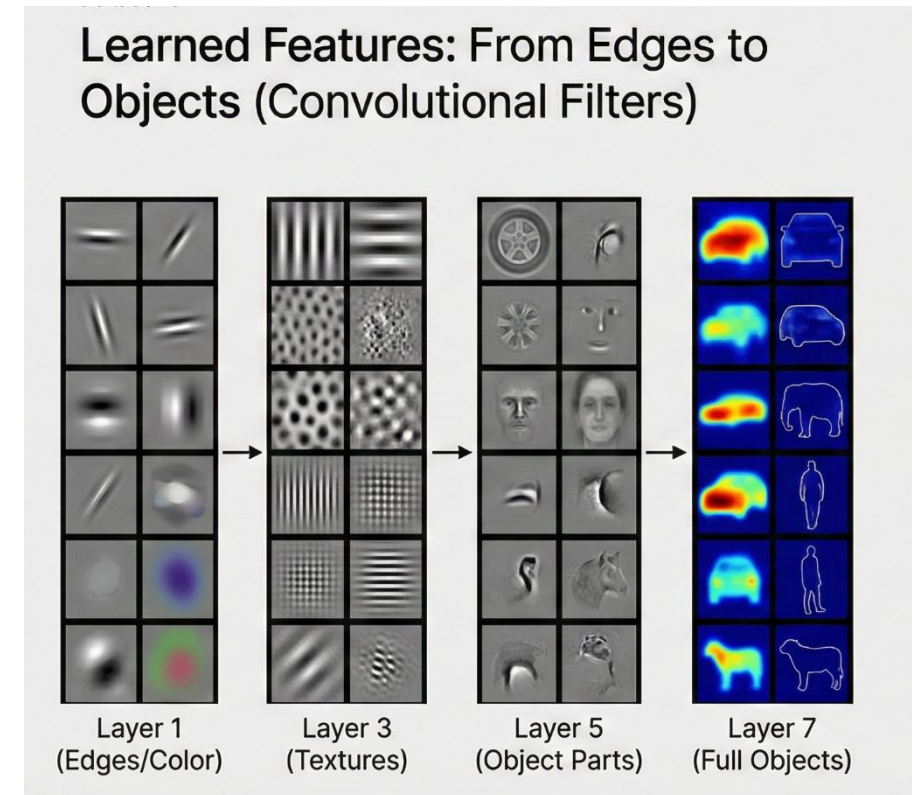
- **Classical Pipeline**
 - Hand-crafted features → Algorithm → Segmentation
 - (Humans design features, algorithm stays fixed)
- **Deep Learning Pipeline**
 - Raw pixels → Neural Network → Segmentation
 - (Network learns features + the segmentation rules)
- **Key Breakthroughs**

Learned Features

Networks learn hierarchical representations automatically:

- Early layers → Edges, simple colors
- Mid layers → Textures, repeated patterns
- Deep layers → Object parts (eyes, wheels)
- Final layers → Full objects (faces, cars)

❖ No manual feature engineering required.



PART 4: DEEP LEARNING ERA

The Deep Learning Revolution - Why It Changed Everything

End-to-End Learning

- One model trained with backpropagation
- Features + segmentation optimized together
- Removes hand-crafted rules

• Scale With Data

- Performance improves with more data.
Classical methods plateau early.

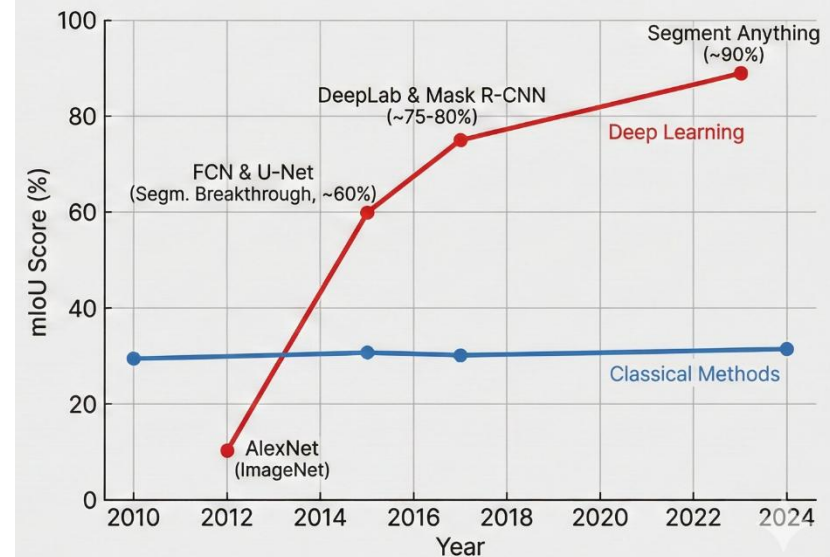
Transfer Learning

- Models pre-trained on massive datasets (ImageNet, COCO)
- Fine-tune on smaller tasks with strong results

What Enabled This?

- **GPU computing** (massive parallel processing)
- **Large datasets** (ImageNet, COCO)
- **Better architectures** (CNNs, skip connections)
- **Improved training techniques** (batch norm, dropout, Adam)

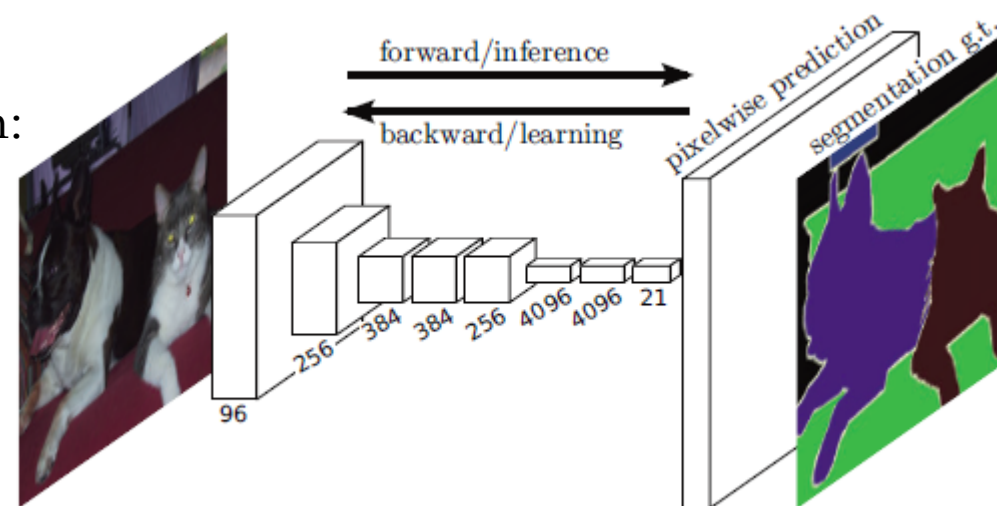
Performance Breakthrough: Pascal VOC mIoU Timeline (2010-2024)



Fully Convolutional Networks (FCN)

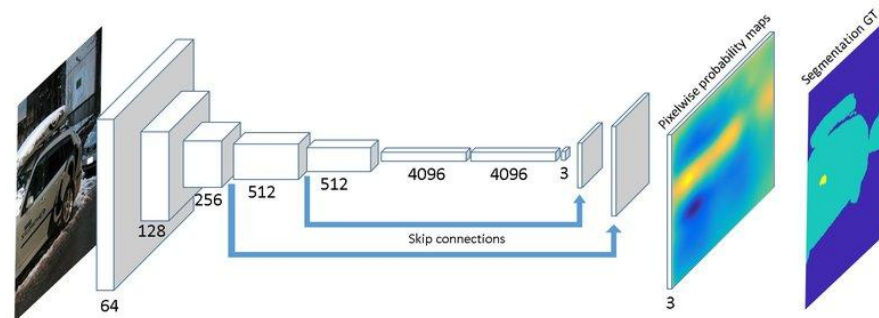
Why Regular CNNs Fail for Segmentation

- Classification networks compress the image too much:
 - Input $\rightarrow$ Convs $\rightarrow$ Pooling $\rightarrow$ **Flatten**
 - Fully connected layers destroy spatial layout
 - Output is a **single label**, not per-pixel predictions
- This makes them useless for dense tasks like segmentation.



The FCN Breakthrough (Long, Shelhamer, Darrell, 2015)

- **Remove Fully Connected Layers**
 - Replace them with **convolutions**, keeping spatial structure intact.
- **Predict a Heatmap**
 - Input: $H \times W \times 3$
 - After down sampling: $H/32 \times W/32 \times \text{channels}$
 - Apply 1×1 conv per location $\rightarrow$ **class scores per pixel**
 - Up sample back to $H \times W$ to produce the final mask



Fully Convolutional Networks (FCN)

Key Innovations

Fully Convolutional Architecture

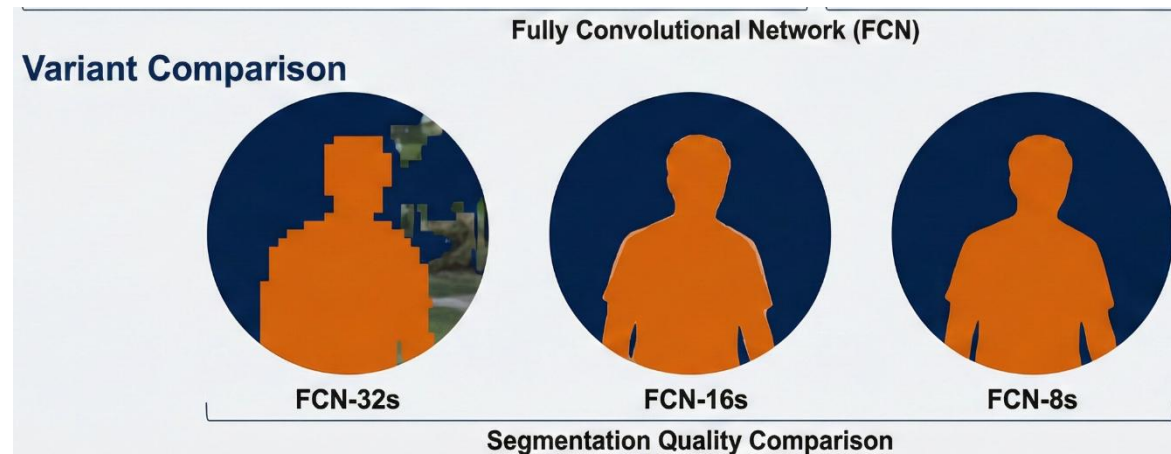
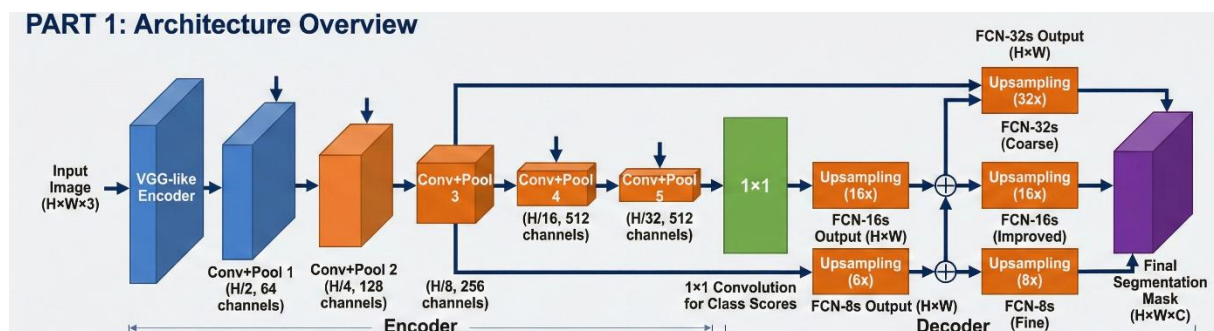
- No dense layers
- Preserves spatial arrangement
- Works on any image size

Skip Connections

- Combine:
 - **Deep, semantic features** (coarse)
 - **Shallow, spatial features** (fine)
- This improves boundary quality.
- **Variants:**
 - **FCN-32s**: coarse result
 - **FCN-16s**: adds pool4 skip
 - **FCN-8s**: adds pool3 skip → **best boundaries**

Transfer Learning

- Initialize encoder with pretrained **VGG/ResNet** to stabilize training.



U-Net – The Medical Imaging Favorite

Why Medical Imaging Needed Something New

- Medical images often have:
 - Very **little training data** (hundreds, not millions)
 - **Life-critical boundaries** (tiny errors matter)
 - Mostly **grayscale scans** (CT, MRI, microscopy)
 - Strong need for **pixel-accurate** segmentation
- U-Net was designed specifically to solve these constraints.

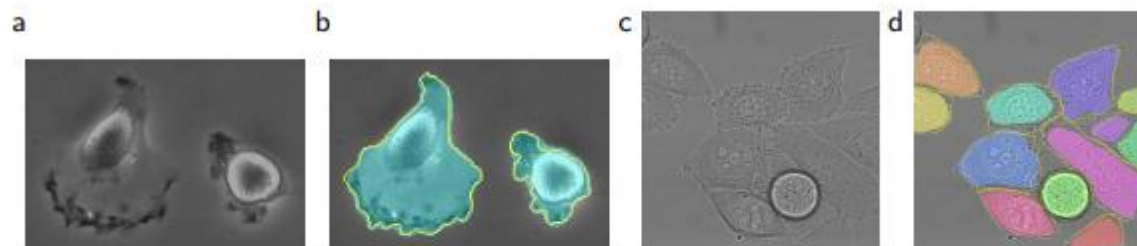
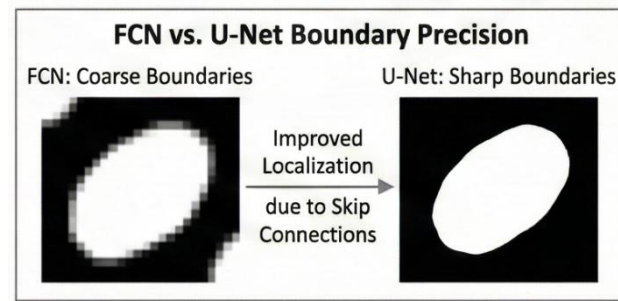
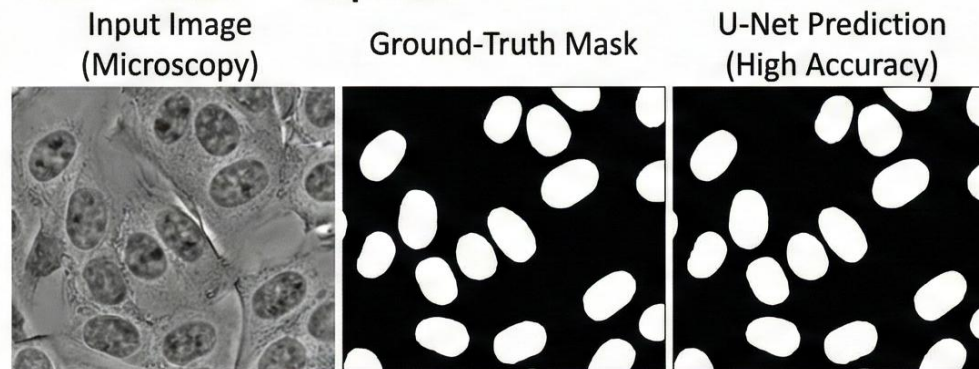


Fig. 4. Result on the ISBI cell tracking challenge. (a) part of an input image of the “PhC-U373” data set. (b) Segmentation result (cyan mask) with manual ground truth (yellow border) (c) input image of the “DIC-HeLa” data set. (d) Segmentation result (random colored masks) with manual ground truth (yellow border).

PART 2: Results Comparison



U-Net – The Medical Imaging Favorite

- U-Net Architecture (Ronneberger et al., 2015)

Encoder – Contracting Path

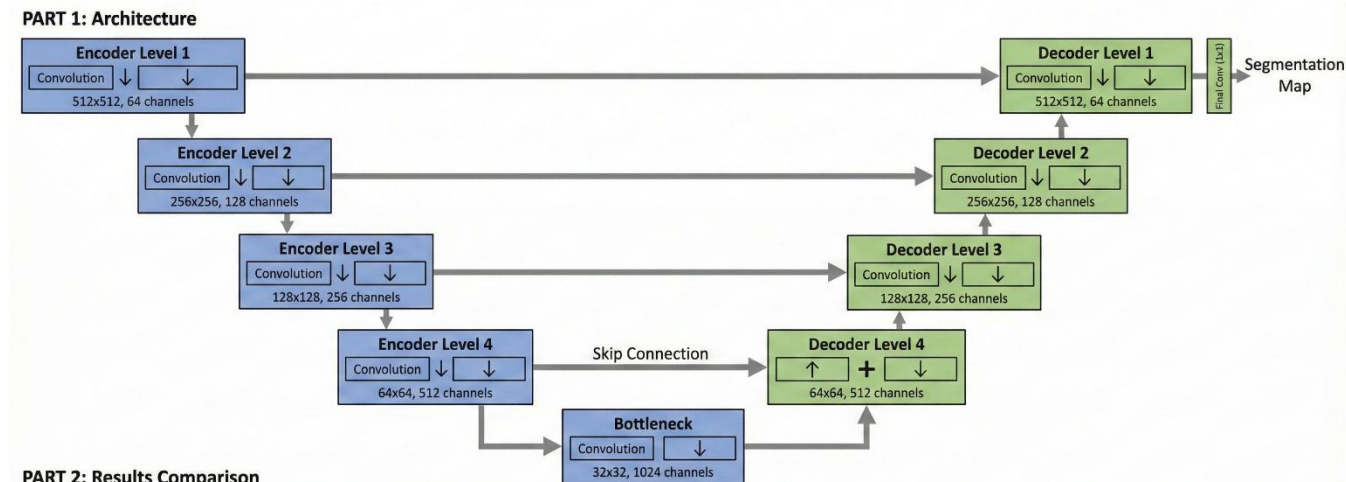
- Repeated blocks of:
 - Convolution → Convolution → MaxPool
 - Extracts context
 - Shrinks spatial resolution

Decoder – Expansive Path

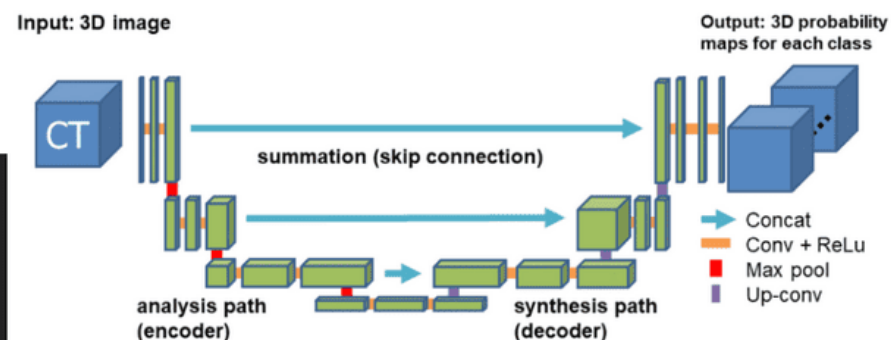
- Repeated blocks of:
 - Up-convolution
 - Concatenation with encoder feature maps
 - Convolution → Convolution
 - Recovers spatial precision

Bottleneck

- Deepest layer combining:
 - High-level semantic representation
 - Smallest spatial resolution



```
def forward(self, x):  
    x1 = self.inc(x)  
    x2 = self.down1(x1)  
    x3 = self.down2(x2)  
    x4 = self.down3(x3)  
    x5 = self.down4(x4)  
    x = self.up1(x5, x4)  
    x = self.up2(x, x3)  
    x = self.up3(x, x2)  
    x = self.up4(x, x1)  
    x = self.outc(x)  
    return x
```



U-Net – The Medical Imaging Favorite

Key Features

Symmetric U-Shape

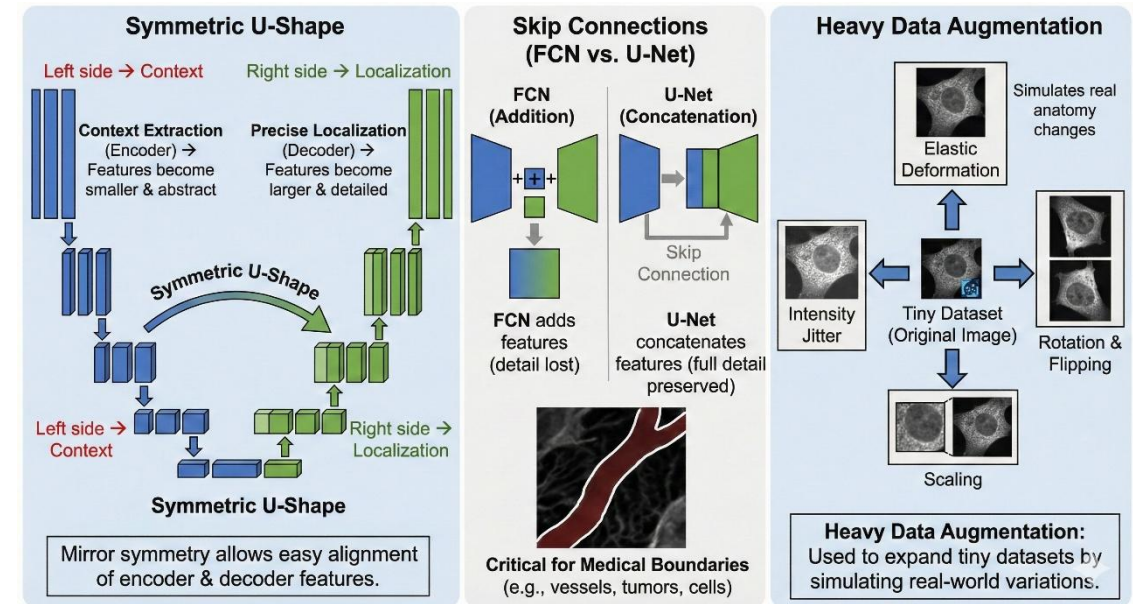
- Left side → **Context extraction**
- Right side → **Precise localization**
- The mirror symmetry makes it easy to align encoder and decoder features.

Skip Connections (Better Than FCN's)

- FCN adds features
- **U-Net concatenates features**
- This preserves **full low-level detail**
- Critical for medical boundaries like vessels, tumors, cells

Heavy Data Augmentation

- Used because datasets are tiny:
 - Elastic deformation (simulates real anatomy changes)
 - Rotation and flipping
 - Scaling
 - Intensity jitter



Weighted Loss for Hard Regions

Gives extra weight to:

- Object borders
 - Thin structures
 - Touching cells
- Helps prevent merging or missing fine details.

U-Net – The Medical Imaging Favorite

Why U-Net Became the Standard

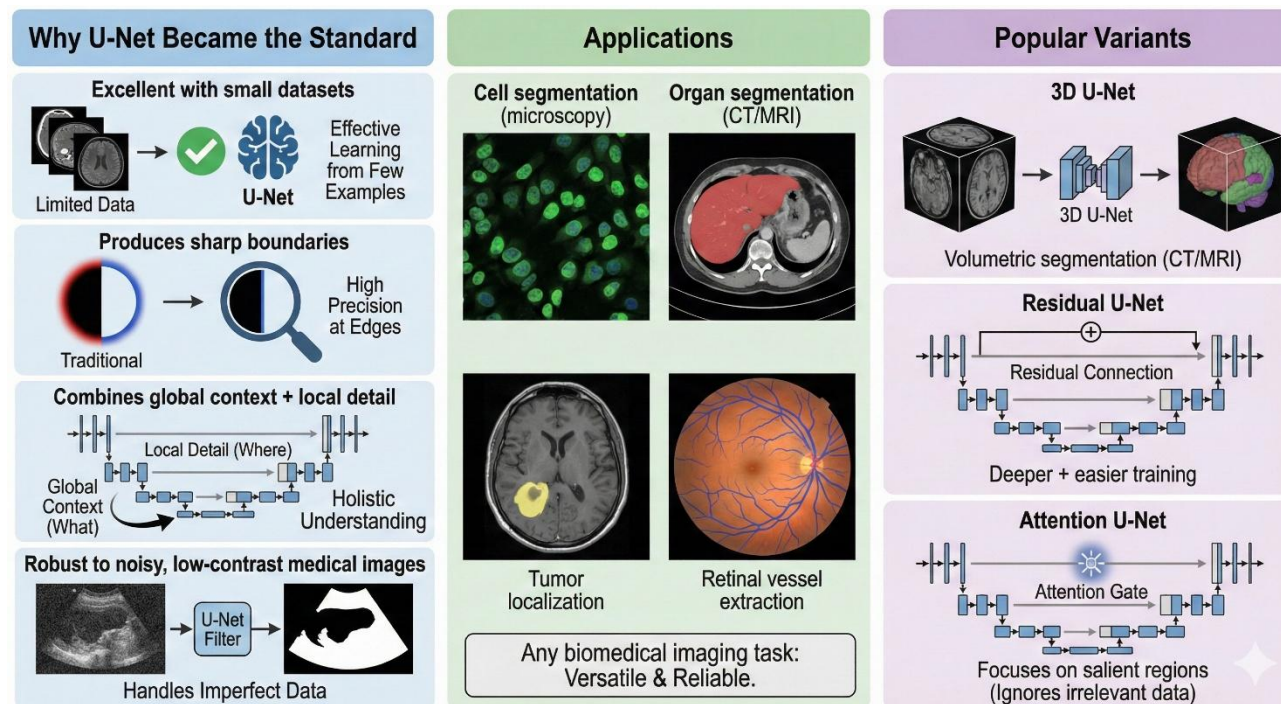
- Excellent with **small datasets**
- Produces **sharp boundaries**
- Combines **global context + local detail**
- Robust to noisy, low-contrast medical images

Applications

- Cell segmentation (microscopy)
- Organ segmentation (CT/MRI)
- Tumor localization
- Retinal vessel extraction
- Any biomedical imaging task

Popular Variants

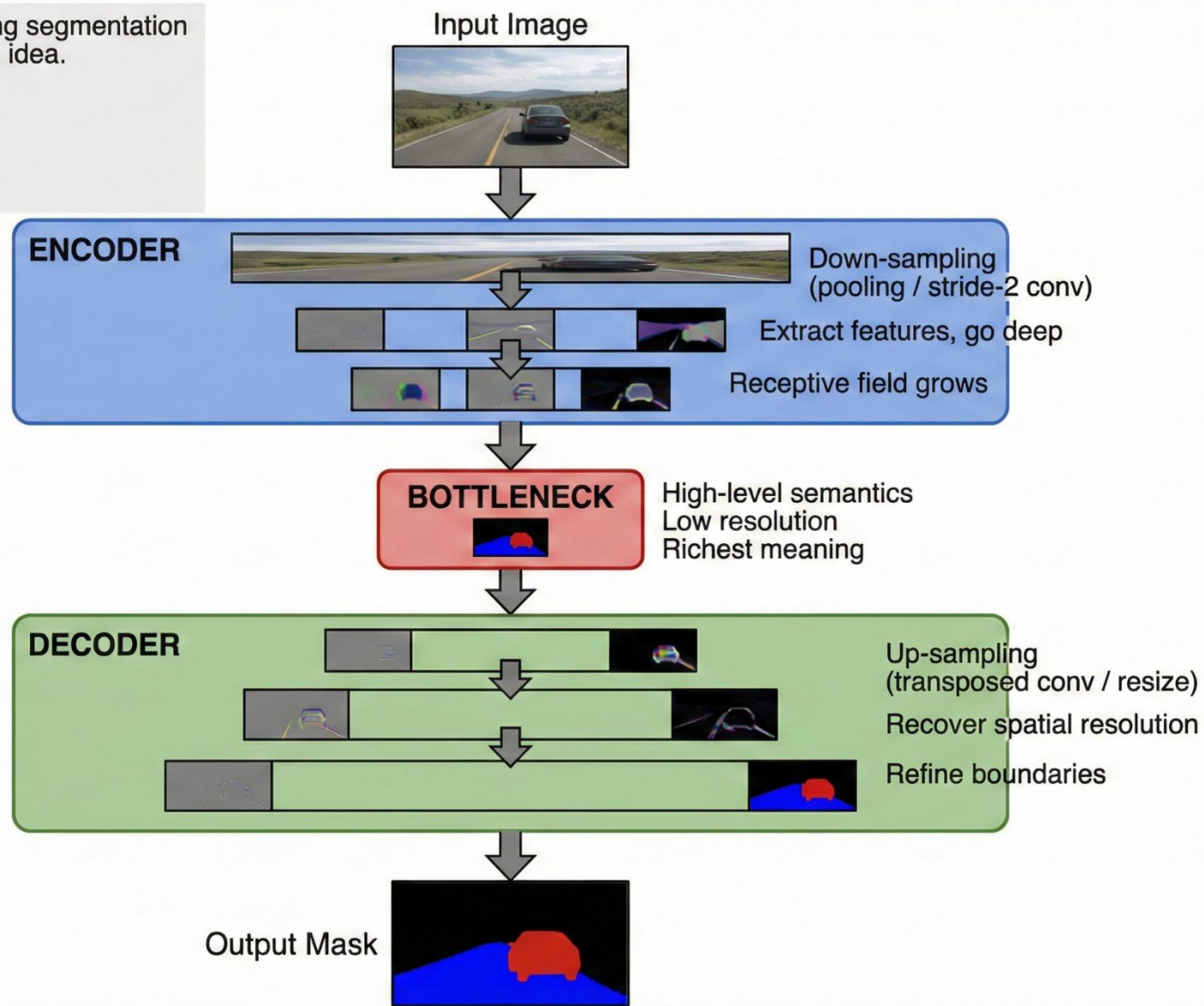
- **3D U-Net** – volumetric segmentation for CT/MRI
- **Residual U-Net** – deeper + easier training
- **Attention U-Net** – focuses on salient regions



Encoder-Decoder Architectures

Why This Pattern Exists: All strong segmentation models follow the same backbone idea.

- FCN
- U-Net
- SegNet
- DeepLab
- PSPNet



Encoder-Decoder Architectures

Key Design Choices (What You Trade Off)

Encoder

- VGG / ResNet / EfficientNet / MobileNet
- Trade-off: Depth vs speed vs memory

Decoder

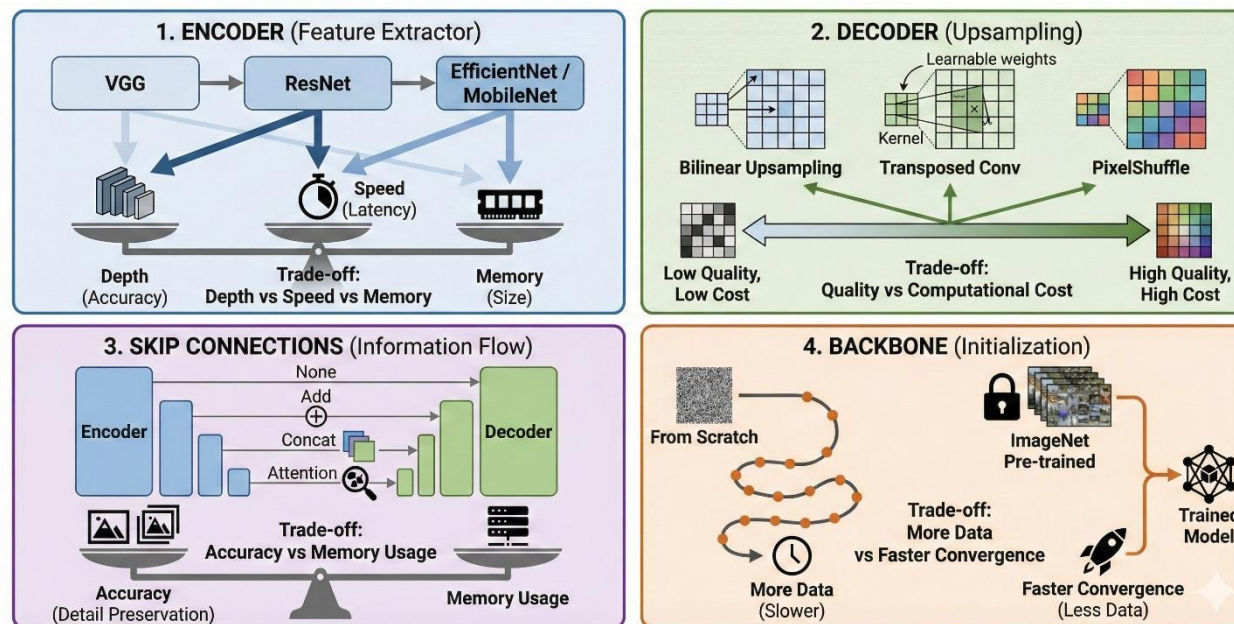
- Bilinear up-sampling / Transposed Conv / PixelShuffle
- Trade-off: Quality vs computational cost

Skip Connections

- None / Add / Concat / Attention
- Trade-off: Accuracy vs memory usage

Backbone

- From scratch / ImageNet pre-trained
- Trade-off: More data vs faster convergence



Encoder–Decoder Architectures

Representative Architectures

- **SegNet**

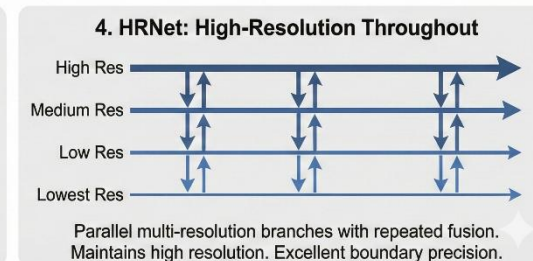
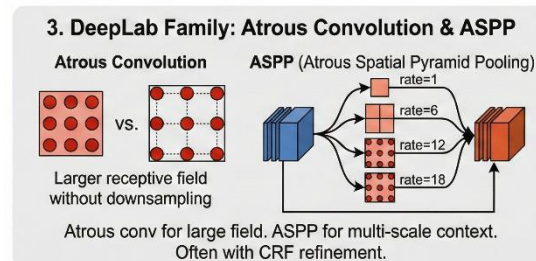
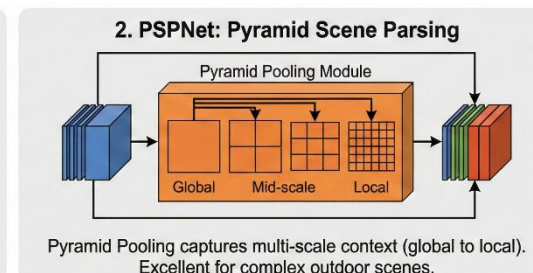
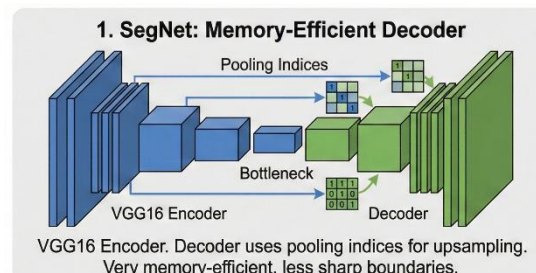
- Encoder = VGG16
- Decoder uses **pooling indices** instead of learnable upsampling
- Very memory-efficient, but not as sharp as U-Net

- **PSPNet (Pyramid Scene Parsing)**

- Introduced **Pyramid Pooling Module**
- Captures:
 - Global context
 - Mid-scale context
 - Local detail
- Works extremely well for complex outdoor scenes (Cityscapes).

- **DeepLab Family**

- **Atrous (Dilated) convolutions** → large receptive field without down-sampling
- **ASPP (Atrous Spatial Pyramid Pooling)** → multi-scale context
- Often used with CRF for refinement (older versions)



HRNet

- Holds **high resolution throughout** the network
- Parallel branches at:
 - High res
 - Medium res
 - Low res
- Fuses them repeatedly
- Excellent boundary precision

DeepLabv3+ — State-of-the-Art Semantic Segmentation

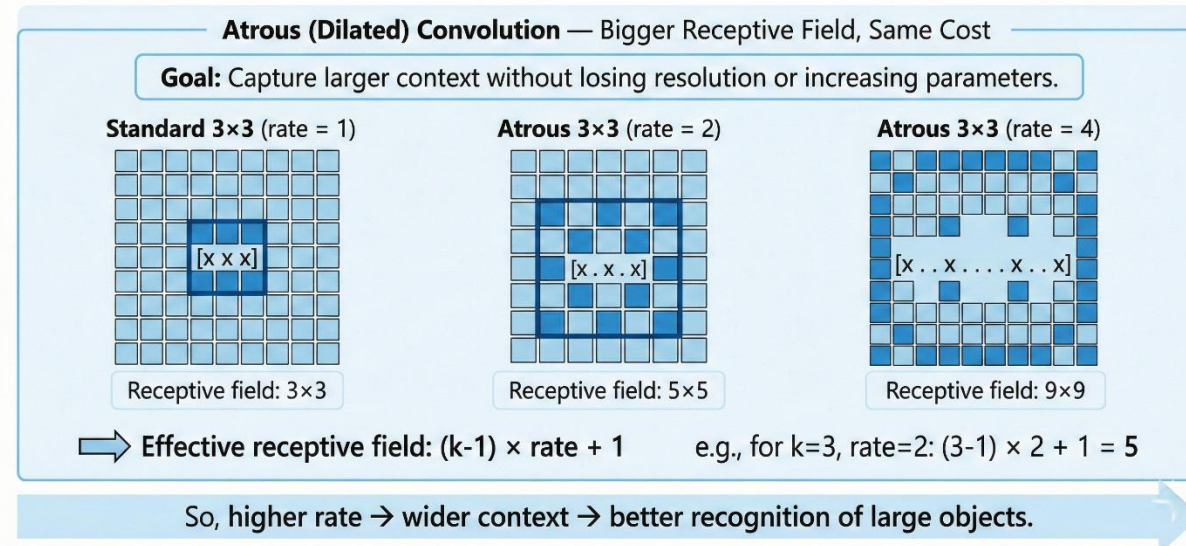
Why It Matters

- DeepLabv3+ combines dilated convolutions, ASPP, and a clean decoder, giving excellent accuracy on complex datasets (Cityscapes, Pascal VOC).



Atrous (Dilated) Convolution — Bigger Receptive Field, Same Cost

- Goal: capture larger context without losing resolution or increasing parameters.
- Standard 3×3 (rate = 1): $[x \ x \ x]$
- Atrous 3×3 (rate = 2): $[x \ . \ x \ . \ x]$
- Atrous 3×3 (rate = 4): $[x \ . \ . \ x \ . \ . \ x]$
- Effective receptive field:
 $(k - 1) \times rate + 1$
- So, higher rate $\rightarrow$ wider context $\rightarrow$ better recognition of large objects.



DeepLabv3+ — State-of-the-Art Semantic Segmentation

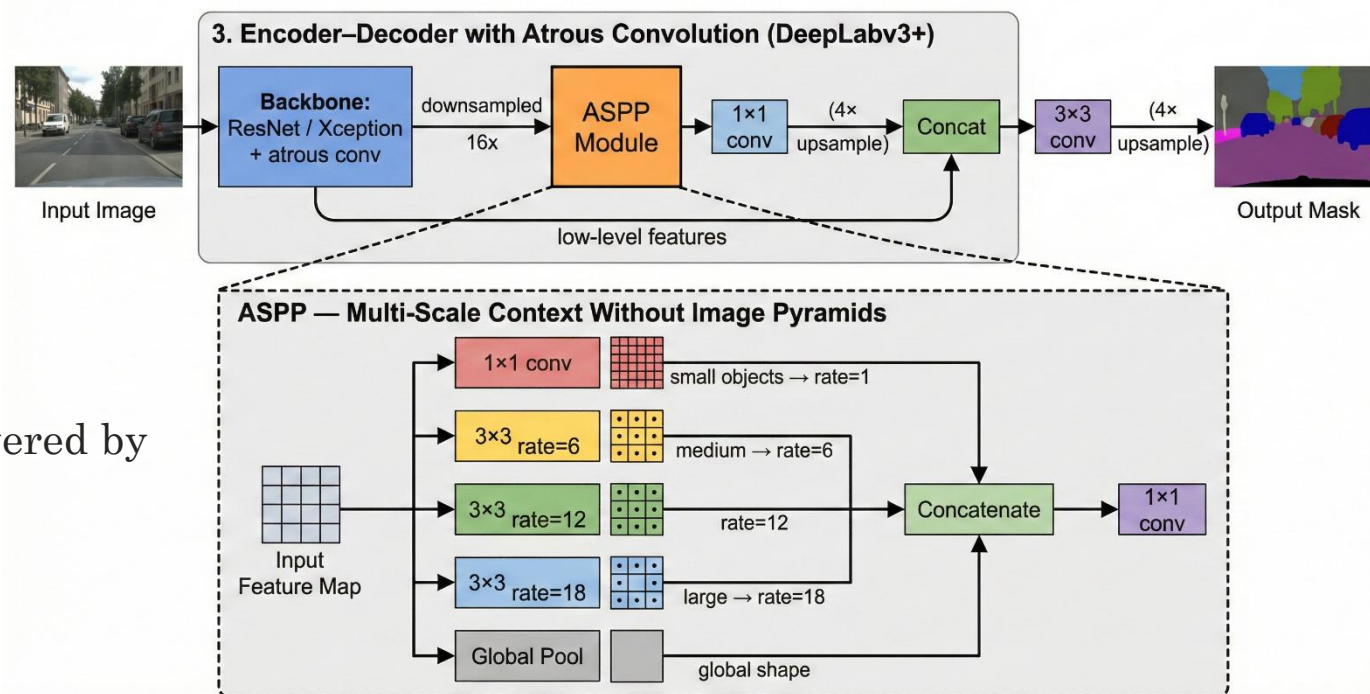
ASPP — Multi-Scale Context Without Image Pyramids

- ASPP does **simultaneous multi-scale analysis**:
 - small objects $\rightarrow$ rate=1
 - medium $\rightarrow$ rate=6, 12
 - large $\rightarrow$ rate=18
 - global shape $\rightarrow$ global pooling
- **Encoder–Decoder with Atrous Convolution (DeepLabv3+)**
 - A refined U-Net-like structure but powered by atrous convs.

Why It Works

- Multi-scale understanding (ASPP)
- Huge receptive field without blurring
- Decoder restores sharp edges
- Strong performance on fine boundaries (roads, poles, pedestrians)

DeepLabv3+ Architecture with Atrous Spatial Pyramid Pooling (ASPP)



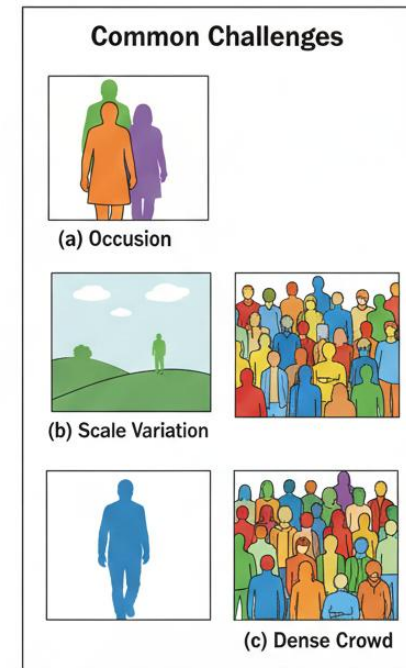
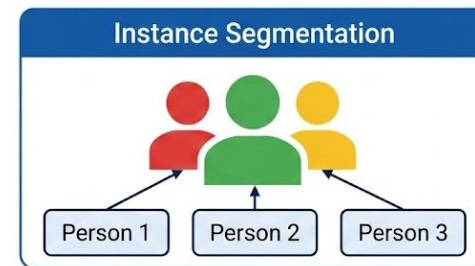
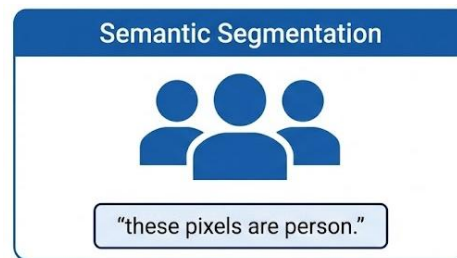
Instance Segmentation — Distinguishing Individuals

What Problem Does It Solve?

- Semantic segmentation only says “**these pixels are person.**”
Instance segmentation must say:
 - Person 1
 - Person 2
 - Person 3
- Even when they overlap or vary in size.

Challenges

- Changing instance count (1 → 100+)
- Occlusions (one object covering another)
- Scale variation (tiny vs huge objects)
- Crowded scenes



Instance Segmentation — Distinguishing Individuals

Two Approaches

- **Top-Down (Detection → Masking)**

- **Steps:**

- Detect objects (bounding boxes)
 - Predict a precise mask inside each box

- **Example: Mask R-CNN**

- Strong accuracy
 - Good for overlapping objects
 - Slower for large numbers of instances

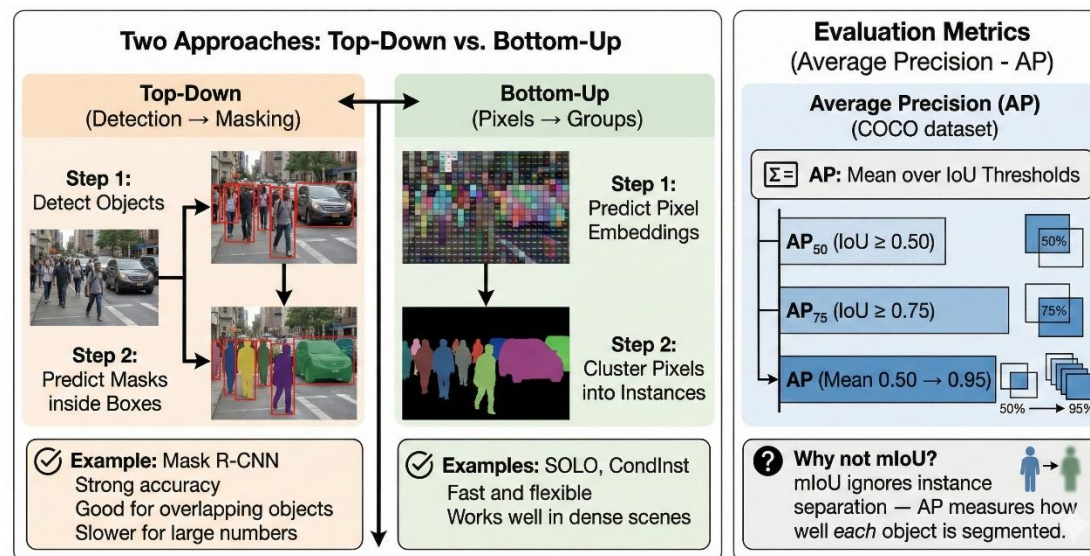
- **Bottom-Up (Pixels → Groups)**

- **Steps:**

- Predict pixel embeddings/features
 - Cluster pixels into separate instances

- **Examples: SOLO, CondInst**

- Fast and flexible
 - Works well in dense scenes



Evaluation Metrics

Average Precision (AP)

Used in COCO dataset:

- **AP₅₀:** IoU ≥ 0.50
- **AP₇₅:** IoU ≥ 0.75
- **AP:** Mean over IoU 0.50 → 0.95

Why not mIoU?

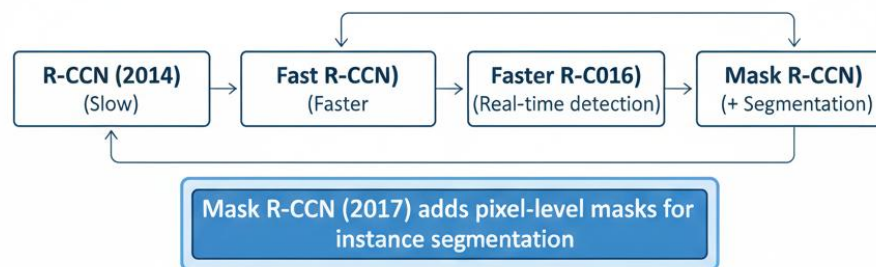
mIoU ignores instance separation — AP measures how well **each object** is segmented.

Mask R-CNN: The Standard for Instance Segmentation

Evolution (How We Got Here)

- **R-CNN (2014):** Accurate but slow
- **Fast R-CNN (2015):** Faster training
- **Faster R-CNN (2016):** Real-time proposals (RPN)
- **Mask R-CNN (2017):** Adds **pixel-level masks** → instance segmentation champion

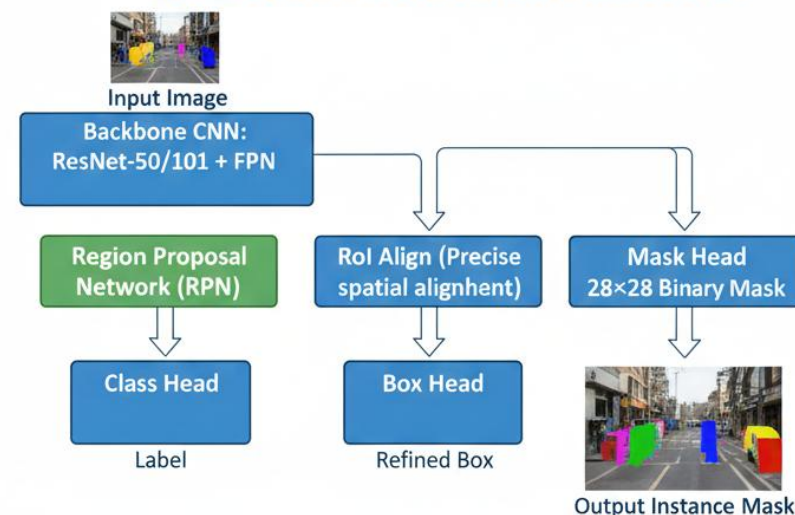
Evolution of R-CCN Family



Architecture Overview

- **Input Image**
 - **Backbone (ResNet-50/101 + FPN)**
 - **Region Proposal Network (RPN)**
 - **RoIs**
 - **RoI Align**
 - **Three Parallel Heads:**
 - **Class Head:** Predicts object category
 - **Box Head:** Refines bounding box
 - **Mask Head:** Predicts **28×28 binary mask** per instance

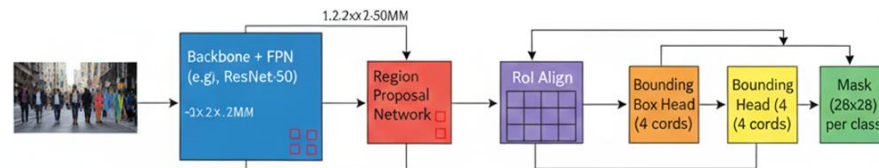
Mask R-CCN Architecture



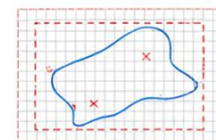
Mask R-CNN: The Standard for Instance Segmentation

Key Components

- **Feature Pyramid Network (FPN)**
 - Combines high-resolution + high-semantic features
 - Gives strong multi-scale detection
- **Region Proposal Network (RPN)**
 - Generates $\sim 1,000$ candidate object regions
 - Fast and fully convolutional
- **RoI Align (Major Upgrade)**
 - **Problem (RoI Pool):**
 - Rounds coordinates $\rightarrow$ misalignment $\rightarrow$ bad masks
 - **Fix (RoI Align):**
 - No rounding
 - Uses **bilinear interpolation**
 - Results in **sub-pixel accuracy**
 - Essential for high-quality masks

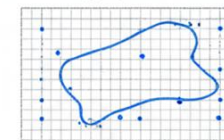


RoI Pool (Old Method)



☹ Rounding leads of misalignment

RoI Align (Mask R-CHN)



✓ Bilinear interpolation for sub-pixel precision



Instance segmentation with bounding boxes and masks.
Handles occlusion successfully.

Mask Branch

- Runs **in parallel** to class + box branches
- FCN-based small decoder
- Outputs **28×28 mask** for each class
- Refined to full resolution at the end

YOLO Evolution: From Detection to Segmentation

Core Idea

- **YOLO = You Only Look Once**

A **single-stage** network that predicts bounding boxes, classes, and (now) masks **in one forward pass**.

No region proposals. No multi-stage processing. Pure speed.

- **Evolution Timeline (2016 → 2024)**

- YOLOv1 (2016) — First real-time detector (45 FPS)
- YOLOv2 (2017) — Anchor boxes, batch norm
- YOLOv3 (2018) — Multi-scale predictions, Darknet-53
- YOLOv4 (2020) — CSPDarknet53 + bag-of-freebies
- YOLOv5 (2020) — PyTorch rewrite (industry adoption explodes)
- YOLO-NAS (2023) — Neural Architecture Search optimized
- YOLOv8 (2023) — Unified framework for detect/classify/segment
- YOLOv8-Seg (2023) — **Instance segmentation** added
- YOLOv9 / YOLOv10 / YOLOv11 (2024) — Efficiency + accuracy upgrades

YOLO: You Only Look Once

Core Idea



**No Region Proposals.
No Multi-Stage Processing.
Pure Speed. ⚡**

Evolution Timeline (2016 → 2024)



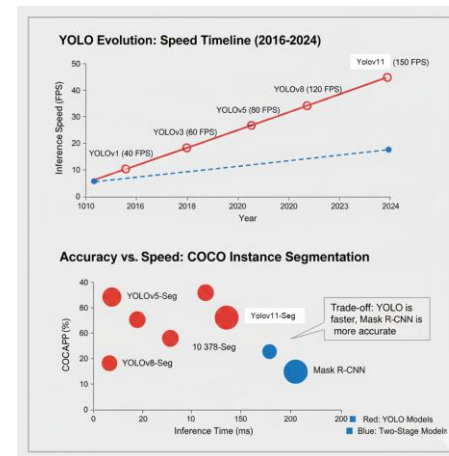
YOLO Evolution: From Detection to Segmentation

Why YOLO for Segmentation?

- **Fast:** 30–60 FPS on a mid-range GPU
- **Lightweight:** Runs on laptops, embedded devices, even mobile
- **Simple:** One network, one forward pass
- **Practical:** Ideal for real-time tasks (CCTV, robotics, drones, AR)

YOLO vs. Mask R-CNN (Direct Comparison)

Feature	Mask R-CNN	YOLOv8-Seg
Architecture	Two-stage	One-stage
Speed (FPS)	5–7	30–60
Accuracy (COCO AP)	~37.1%	~36%
Complexity	High	Medium
Deployment	Server	Edge/Mobile



Trade-off:

- YOLO is **much faster**, slightly less accurate.
Perfect for **real-time** systems where latency matters more than 1–2 AP.

YOLOv8 Segmentation: Architecture Overview

Pipeline

Input (640×640×3)

→ Backbone: CSPDarknet

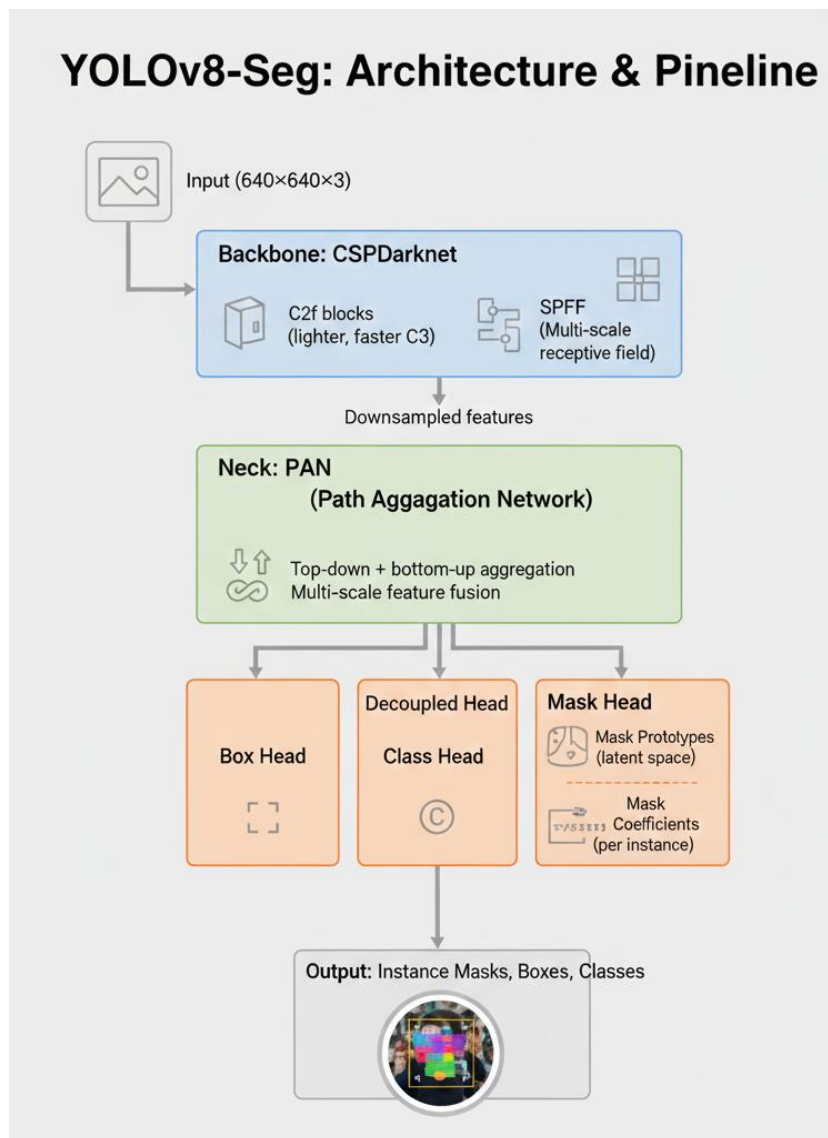
- C2f blocks (lighter, faster C3)
- SPPF for multi-scale receptive field

→ Neck: PAN

- Top-down + bottom-up aggregation
- Multi-scale feature fusion

→ Decoupled Heads

- **Box Head**
- **Class Head**
- **Mask Head** (prototype + coefficients)



YOLOv8 Segmentation: Architecture Overview

Key Improvements

- **Anchor-Free Detection**

- Predict center + width/height directly.
Simplifies training + reduces anchors.

- **Decoupled Head**

- Separate classification and localization → better gradients, cleaner optimization.

- **C2f Module**

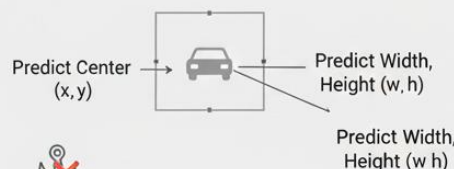
- Hybrid of CSPNet + ELAN:
 - more gradient paths
 - lighter compute
 - better representation learning

- **Mask Prototypes (YOLACT-style)**

- Network predicts **32 global prototypes**
- Each instance predicts **32 coefficients**
- Linear combination → final mask
Massively reduces mask compute per object.

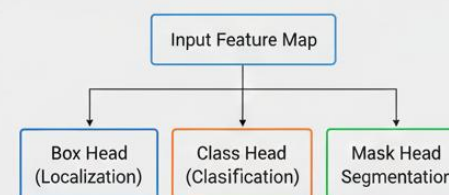
YOLOv8-Seg: Key Improvements

1. Anchor-Free Detection



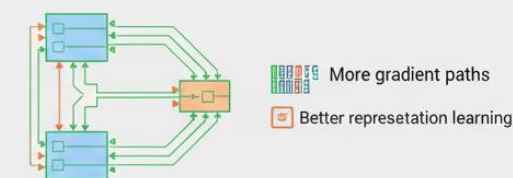
Simplifies training + reduces anchors.

2. Decoupled Head



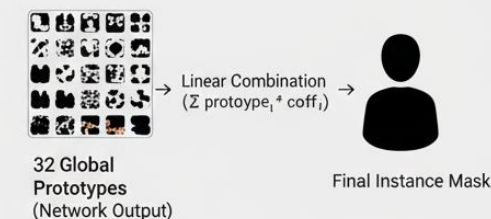
Separate tasks → better gradients, cleaner optimization.

3. C2f Module (Lighter, Faster Backbone)



✓ Lighter compute
Better representation learning

4. Mask Prototypes (YOLACT-style)



Massively reduces mask compute per object.

YOLOv11 Segmentation: Latest Improvements (2024)

What's New in YOLOv11-Seg

• C3k2 Module

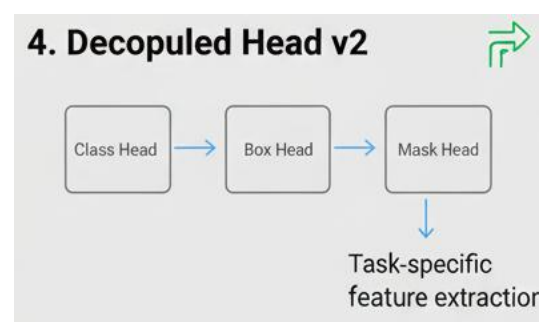
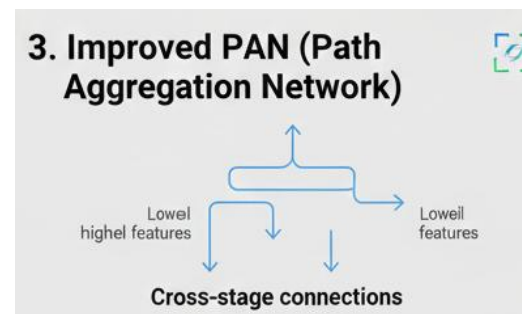
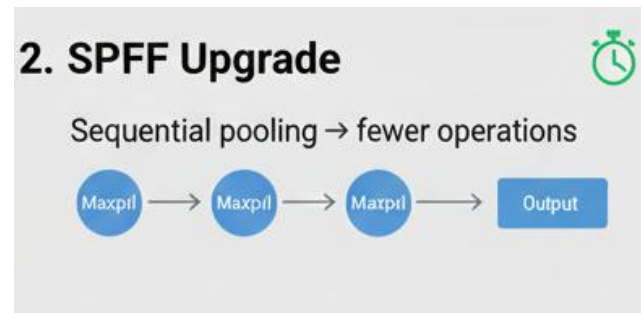
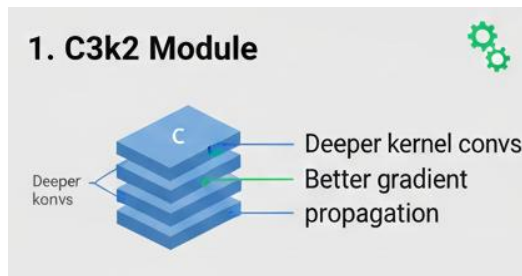
- Enhanced version of C2f
- Deeper kernel convs → stronger multi-scale representation
- Better gradient propagation
- More stable training on high-resolution images

• SPPF Upgrade

- Same receptive field as YOLOv8
- Sequential pooling → fewer operations
- Lower latency, improved throughput

• Improved PAN

- Stronger feature fusion
- Added cross-stage connections
- Dynamic feature selection for small objects



Decoupled Head v2

- Less parameter sharing between heads
- Task-specific feature extraction
- Cleaner gradients for box / class / mask

YOLOv11 Segmentation: Latest Improvements (2024)

Training Improvements

- **Better Augmentation**

- Copy-paste
- Large-scale jitter
- Auto-augment policies

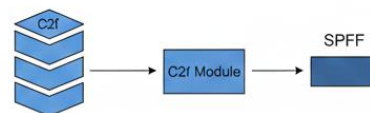
- **Upgraded Loss Functions**

- Varifocal loss → classification
- Alpha-IoU → box regression
- Dice loss → mask prediction

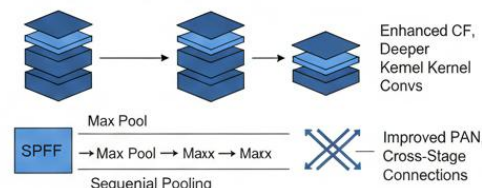
YOLOv8-Seg vs. YOLOv11-Seg: A Comparison

1. Architecture Differences

YOLOv8 Backbone



YOLOv11 Backbone



3. Real-time Instance Segmentation Demo



Performance Gains

Metric	YOLOv8-Seg	YOLOv11-Seg	Δ
AP (box)	52.3%	53.4%	+1.1%
AP (mask)	42.6%	43.9%	+1.3%
Speed	35 FPS	40 FPS	+14%
Params	27.3M	25.9M	-5%

Panoptic Segmentation – Full Scene Understanding

What It Does:

- Panoptic segmentation combines **semantic segmentation** (labeling regions) and **instance segmentation** (separating objects).
Every pixel gets both a **class** and an **instance ID** (if needed).

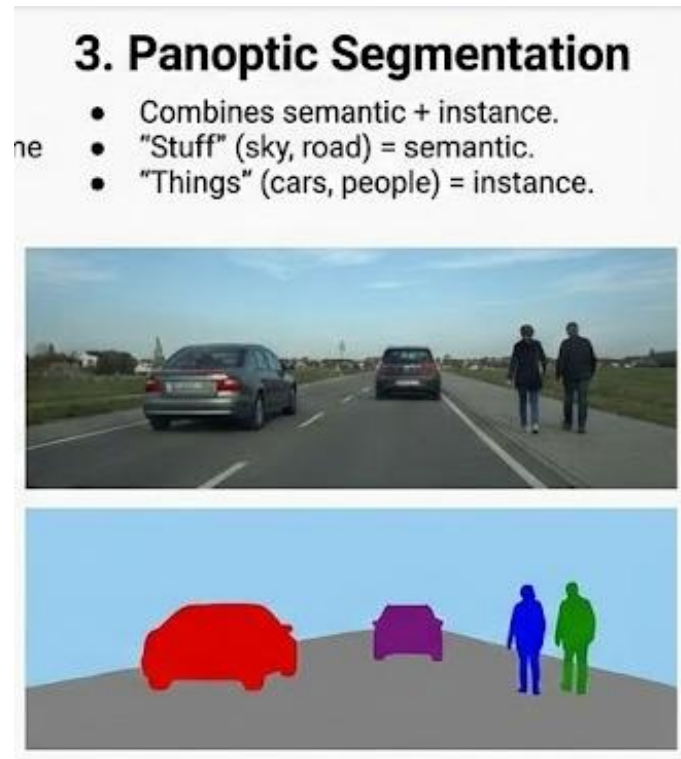
Two Types of Classes

❖ Stuff (No instances)

- Large continuous regions:
 - sky
 - road
 - grass
 - water
- Handled by: **Semantic segmentation**

❖ Things (Countable objects)

- Objects you can separate:
 - person
 - car
 - cat
 - bicycle
- Handled by: **Instance segmentation**



Two Types of Classes

1. Stuff (Uncountable, amorphous)



Large continuous regions (sky, road, grass).
Handled by Semantic Segmentation.

2. Things (Countable, individual)



Individual objects (person, car, cat).
Handled by Instance Segmentation.

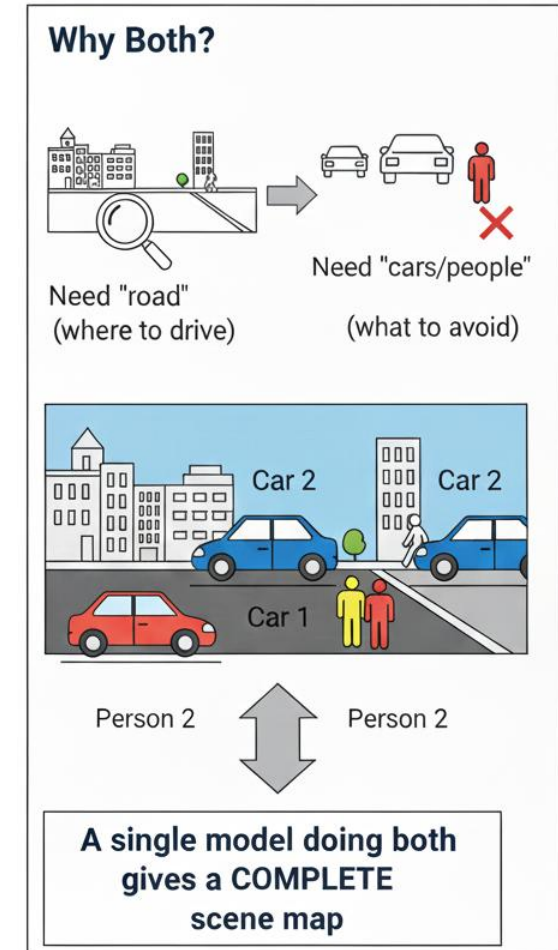
Panoptic Segmentation – Full Scene Understanding

Why Both?

- Example: **Self-driving cars**
 - Need “road” → where to drive
 - Need “cars/people” → what to avoid
- A single model doing both gives a complete scene map.

Panoptic Quality (PQ)

- Measures both segmentation accuracy + detection quality.
- Formula (simple version):
 - High PQ = segments look good **and** all instances detected
 - $PQ = SQ \times RQ$
 - **SQ**: How cleanly shapes are segmented
 - **RQ**: How many segments were correctly found

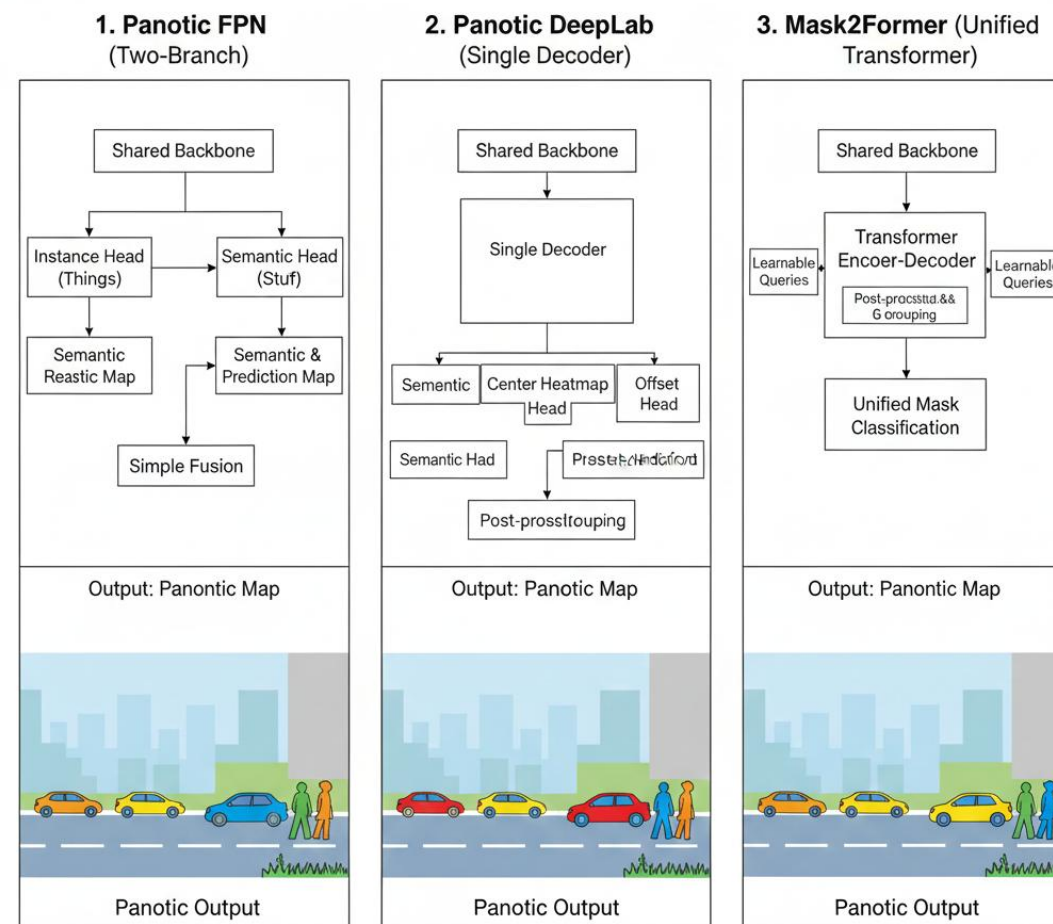


Panoptic Segmentation – Full Scene Understanding

Main Architectures

- **Panoptic FPN (Facebook, 2019)**
 - Mask R-CNN branch (instances)
 - FCN branch (stuff)
 - Combine outputs
- **Panoptic DeepLab (Google, 2020)**
 - Single decoder
 - Predicts: semantic map, center heatmap, offsets
 - Groups pixels into instances
- **MaskFormer / Mask2Former (2021/2022)**
 - Transformer-based
 - One unified mask classification approach
 - Works for semantic, instance, and panoptic

Panoptic Architectures: FPN vs. DeepLab vs. Mask2Former



Transformers for Segmentation - DETR, SegFormer, SAM

Why Transformers Matter in Vision

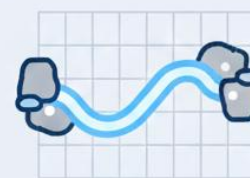
- Transformers changed NLP first (BERT, GPT).
Now they dominate vision because:
 - ❖ They look at the whole image at once
 - ❖ They capture long-range relationships naturally
 - ❖ They remove many hand-designed parts used in older CNN pipelines

1. Whole Image Context



They look at the whole image at once.

2. Long-Range Relationships



They capture long-range relationships naturally

3. Simplified Pipelines



Unified Transformer Model

They remove many hand-designed parts (e.g, manual CNN components).

Transformers for Segmentation - DETR, SegFormer, SAM

DETR (2020) — End-to-End Object Detection

- **What makes it different?**

- No anchors
- No NMS (no post-processing)
- No region-proposal network
- Directly predicts a **set** of objects

- **How it works (simple idea):**

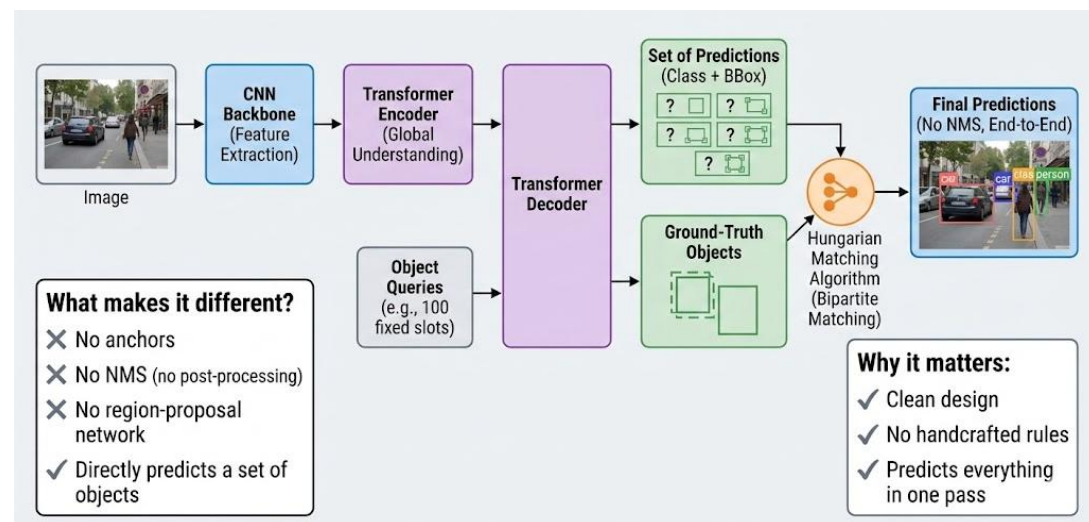
- CNN extracts image features
- Transformer encoder adds global understanding
- Transformer decoder uses **object queries** (100 fixed slots)
- Each query tries to represent one object

- **Matching Step:**

- A Hungarian algorithm pairs predictions with ground-truth objects.

- **Why it matters:**

- Clean design
- No handcrafted rules
- Predicts everything in one pass



Transformers for Segmentation - DETR, SegFormer, SAM

SegFormer (2021) — Efficient & Clean Semantic Segmentation

- **Main idea:**

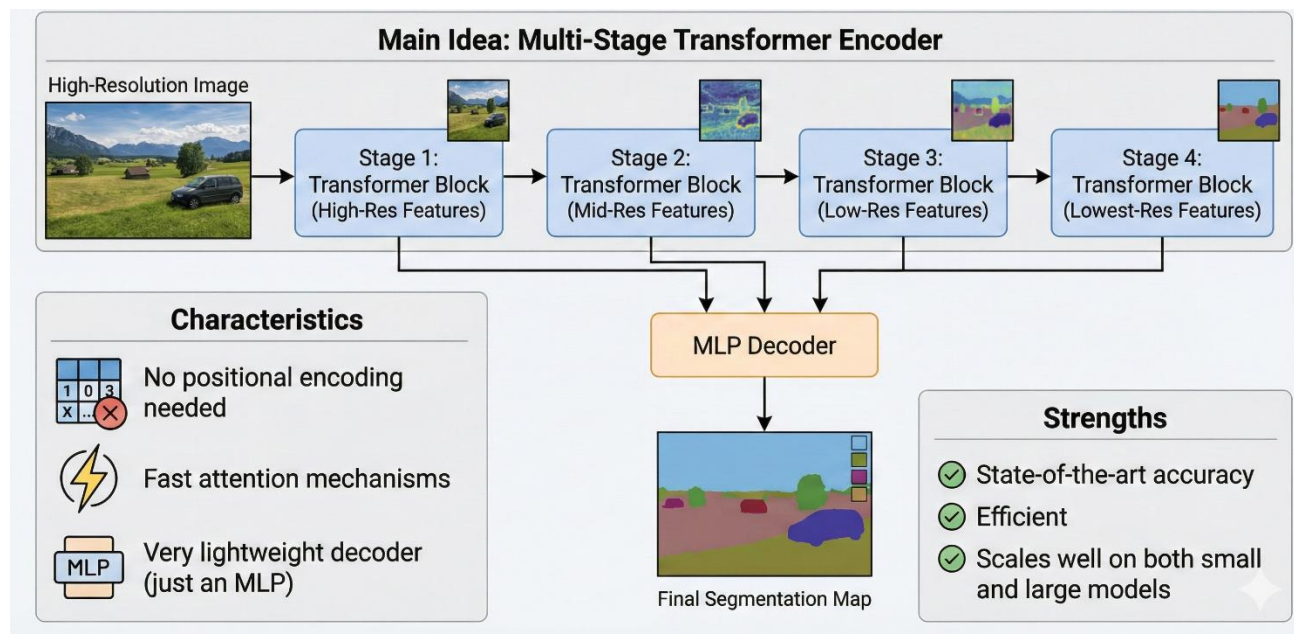
- A transformer encoder that works in **multiple stages** (from high-resolution to low-resolution).

- **Characteristics:**

- No positional encoding needed
- Fast attention mechanisms
- Very lightweight decoder (just an MLP)
- Combines features from all stages for strong segmentation

- **Strengths:**

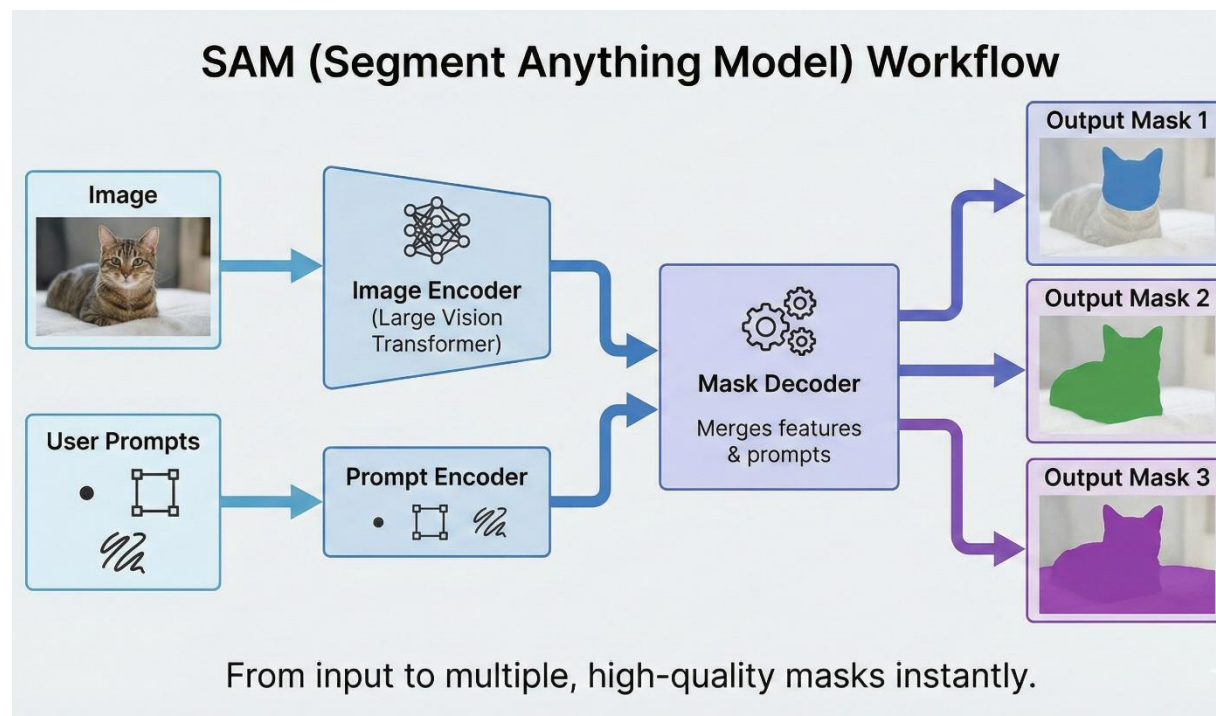
- State-of-the-art accuracy
- Efficient
- Scales well on both small and large models



Transformers for Segmentation - DETR, SegFormer, SAM

SAM – Segment Anything Model (2023)

- **What makes SAM special?**
 - Trained on the **largest segmentation dataset ever** (1B masks)
 - Works on **any image** without training
 - Takes **prompts** (point, box, mask)
 - Predicts multiple high-quality masks instantly
- **How it works (simple flow):**
 - A large Vision Transformer encodes the image
 - A separate encoder processes the user's prompt
 - A mask decoder merges both and outputs several mask options
- **Why it's a breakthrough:**
 - Zero-shot segmentation
 - Works interactively
 - Powerful for medical, robotics, video, AR/VR



Training Deep Segmentation Models — Practical Guide

Dataset Preparation

- **Data Collection**

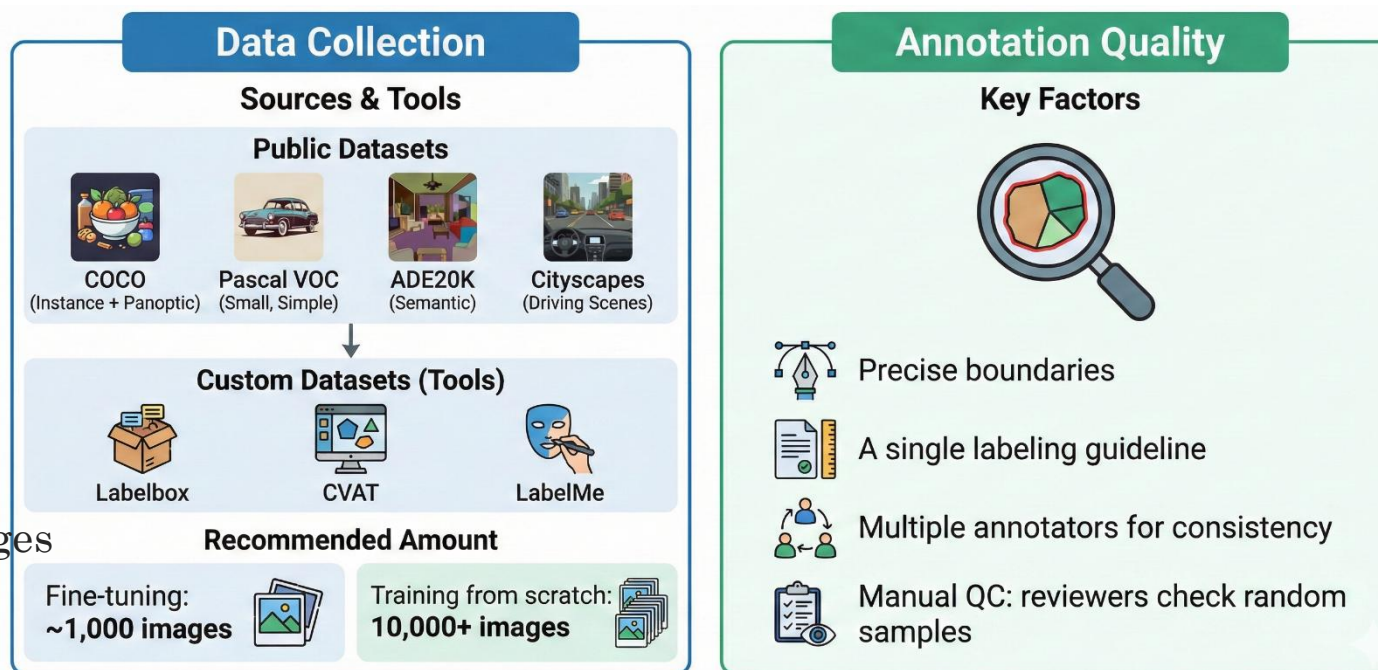
- Use public datasets or build your own:
 - COCO – instance + panoptic
 - Pascal VOC – small, simple
 - ADE20K – semantic segmentation
 - Cityscapes – driving scenes
- For custom datasets → use **Labelbox**, **CVAT**, **LabelMe**.

- **Recommended amount:**

- **Fine-tuning:** ~1,000 images
- **Training from scratch:** 10,000+ images

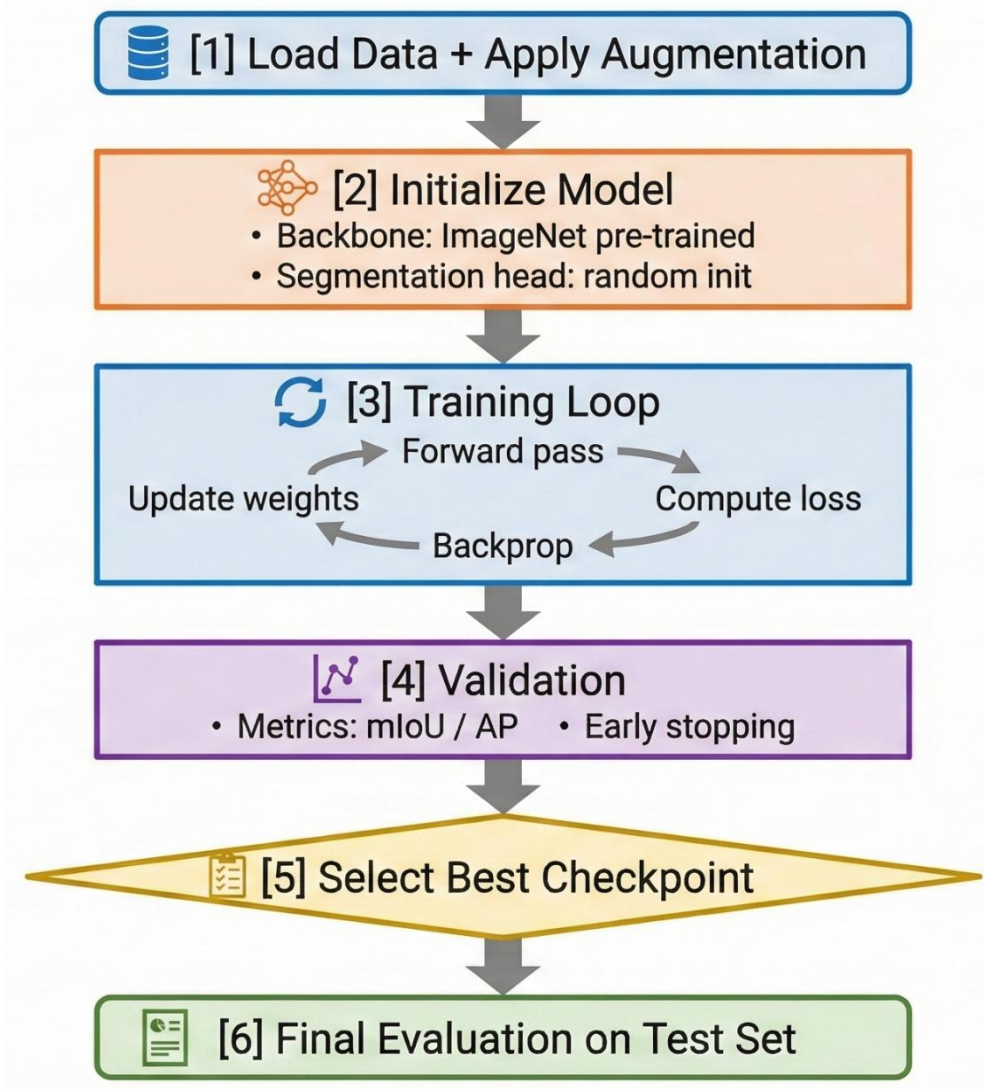
- **Annotation Quality**

- Segmentation quality depends heavily on labels.
 - Precise boundaries
 - A single labeling guideline
 - Multiple annotators for consistency
 - Manual QC: reviewers check random samples



Training Deep Segmentation Models — Practical Guide

Training Pipeline



Training Deep Segmentation Models — Practical Guide

Critical Hyperparameters

Parameter	Typical Value	Notes
Learning Rate	1e-4 → 1e-2	Use LR finder
Batch Size	8–32	Depends on GPU memory
Image Size	512–1024	Larger = better quality
Epochs	50–300	Use early stopping
Optimizer	AdamW, SGD	AdamW for Transformers
LR Schedule	Cosine, Step, Warmup	Warmup = smoother start
Weight Decay	1e-4 → 1e-2	Regularization

Training Deep Segmentation Models — Practical Guide

Data Augmentation

- **Geometric**

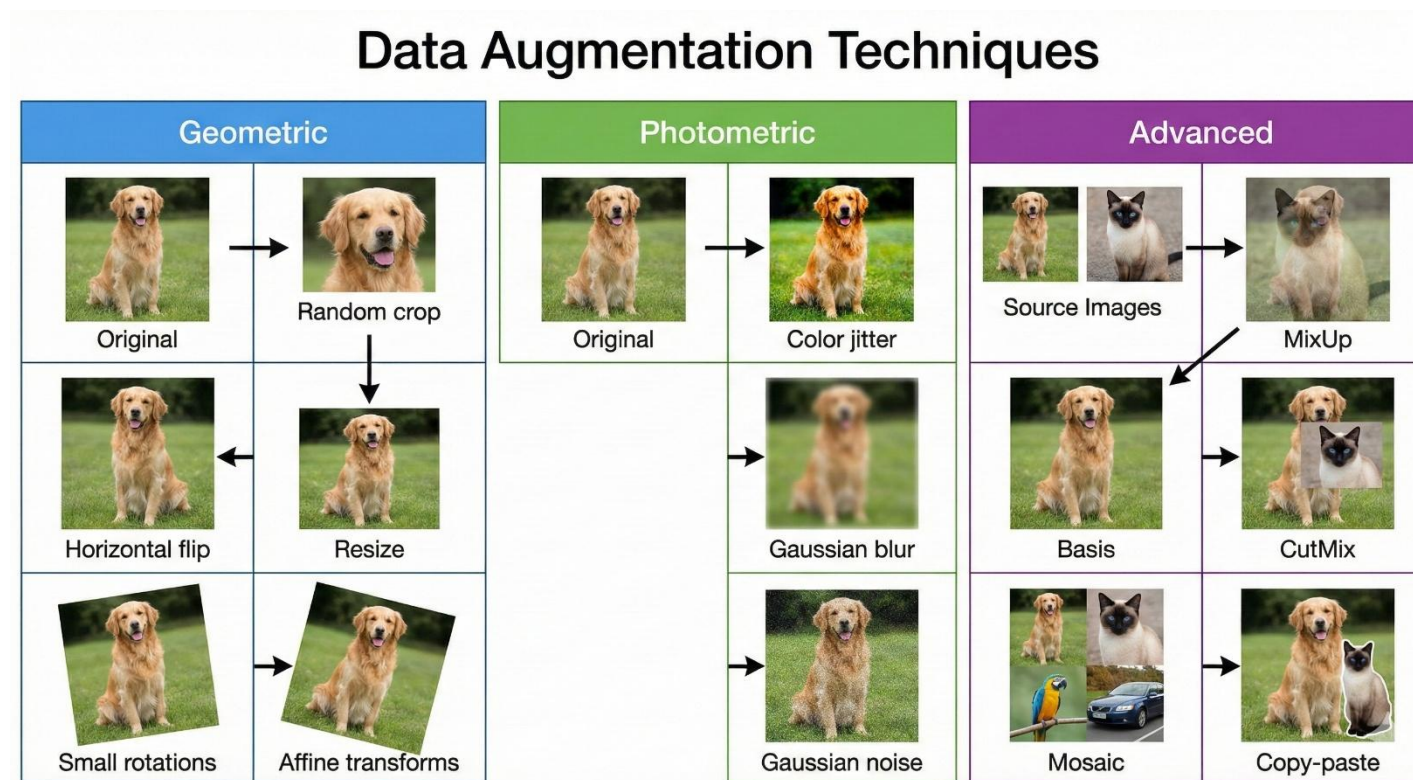
- Random crop
- Resize
- Horizontal flip
- Small rotations
- Affine transforms

- **Photometric**

- Color jitter (brightness/contrast/saturation)
- Gaussian blur
- Gaussian noise

- **Advanced**

- MixUp / CutMix
- Mosaic (YOLO-style)
- Copy-paste (instance segmentation)



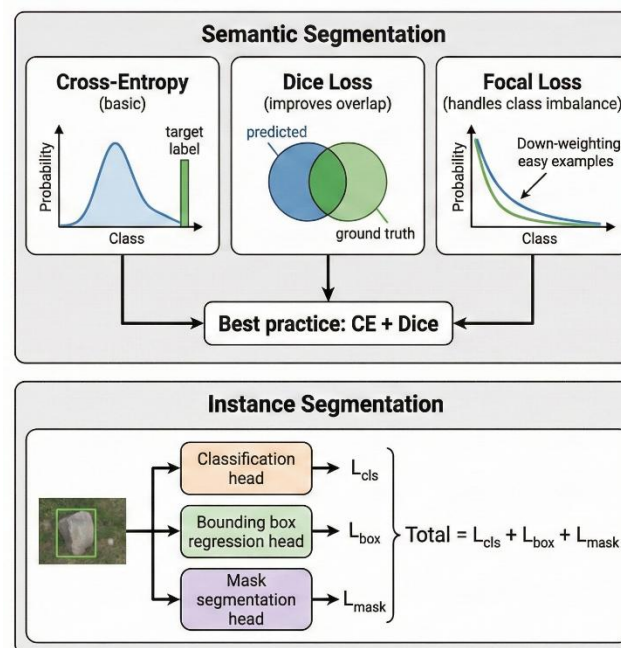
Training Deep Segmentation Models — Practical Guide

Loss Functions

- **Semantic Segmentation**
 - **Cross-Entropy** – basic
 - **Dice Loss** – improves overlap
 - **Focal Loss** – handles class imbalance
- **Best practice: CE + Dice**
- **Instance Segmentation**
 - Standard combined loss:
 - $\text{Total} = L_{\text{cls}} + L_{\text{box}} + L_{\text{mask}}$

Common Pitfalls & Fixes

- **Class imbalance** → weighted loss, oversampling
- **Overfitting** → stronger augmentation, weight decay
- **Small objects missed** → multi-scale training
- **Boundary errors** → boundary-weighted loss
- **LR too high** → unstable training



	Pitfall	Visual Example	Fix
1	Class imbalance		 weighted loss, oversampling
2	Overfitting		 stronger augmentation, weight decay
3	Small objects missed		 multi-scale training
4	Boundary errors		 boundary-weighted loss
5	LR too high		 unstable training

Segmentation Datasets – Benchmarks

Pascal VOC 2012

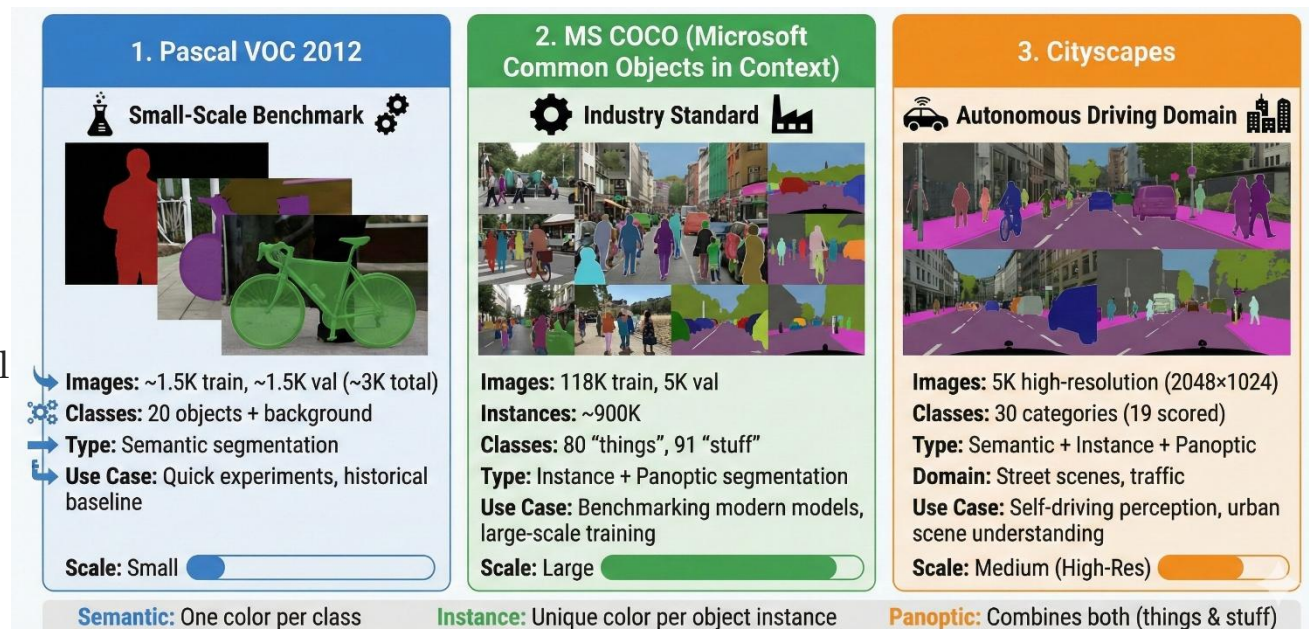
- **Images:** ~1.5K train, ~1.5K val
- **Classes:** 20 objects + background
- **Type:** Semantic segmentation
- **Use Case:** Small-scale benchmark, quick experiments

MS COCO

- **Images:** 118K train, 5K val
- **Instances:** ~900K
- **Classes:** 80 “things”, 91 “stuff”
- **Type:** Instance + panoptic segmentation
- **Use Case:** Industry standard; used for almost all modern models

Cityscapes

- **Images:** 5K high-resolution (2048×1024)
- **Classes:** 30 categories (19 scored)
- **Type:** Semantic + instance + panoptic
- **Domain:** Autonomous driving
- **Use Case:** Street scenes, traffic, self-driving perception



Segmentation Datasets – Benchmarks

ADE20K

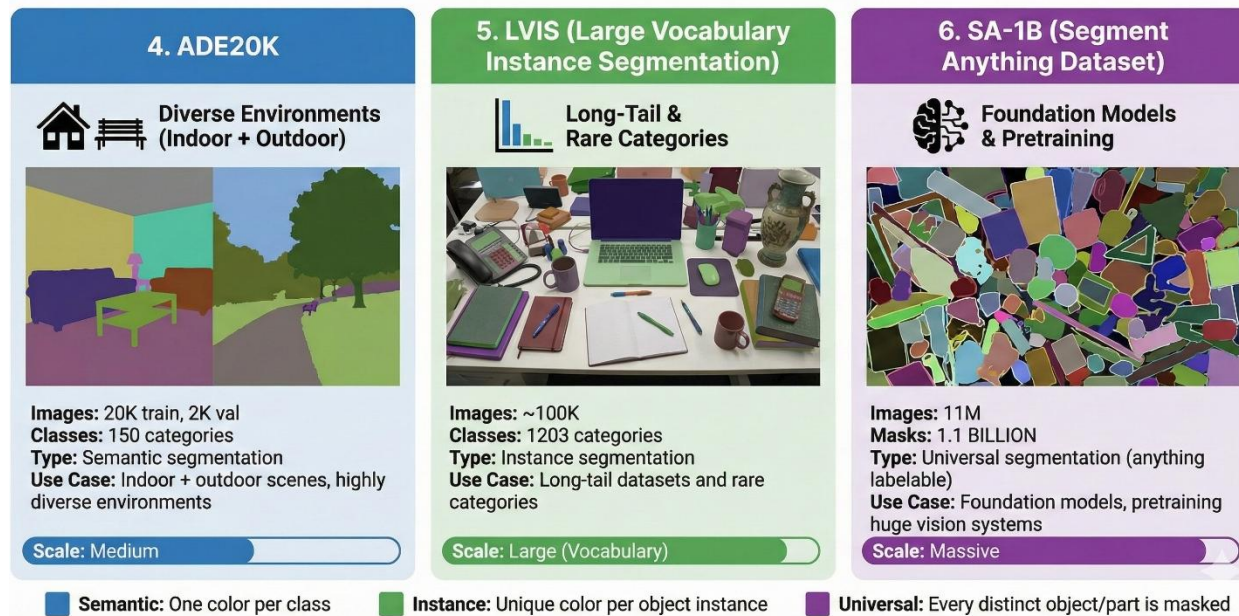
- **Images:** 20K train, 2K val
- **Classes:** 150 categories
- **Type:** Semantic segmentation
- **Use Case:** Indoor + outdoor scenes, highly diverse environments

LVIS (Large Vocabulary)

- **Images:** ~100K
- **Classes:** 1203 categories
- **Type:** Instance segmentation
- **Use Case:** Long-tail datasets and rare categories

SA-1B (Segment Anything Dataset)

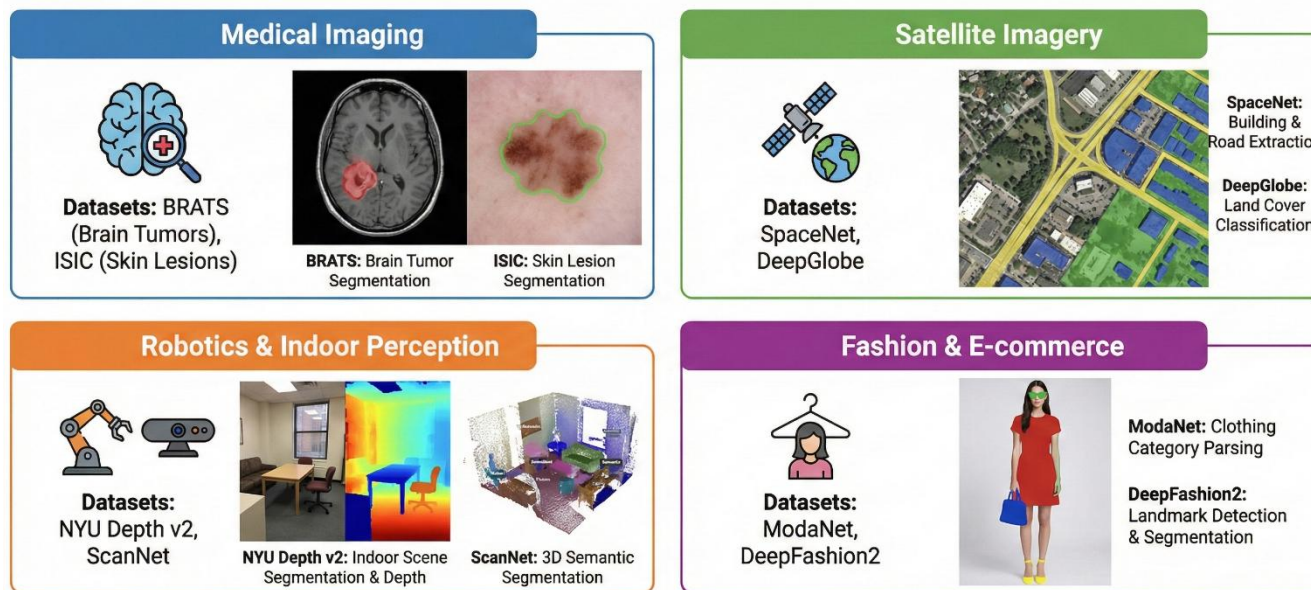
- **Images:** 11M
- **Masks:** 1.1 billion
- **Type:** Universal segmentation (anything labelable)
- **Use Case:** Foundation models, pretraining huge vision systems



Segmentation Datasets – Benchmarks

Domain-Specific Datasets (Examples)

- **Medical:** BRATS (brain tumors), ISIC (skin lesions)
- **Satellite:** SpaceNet, DeepGlobe
- **Robotics:** NYU Depth v2, ScanNet
- **Fashion:** ModaNet, DeepFashion2



These datasets provide specialized data for training segmentation models in specific application domains, enabling precise analysis and automation.

Dataset Comparison Table

Dataset	Size	#Classes	Resolution	Annotation Quality
Pascal VOC	Small	21	Medium	★★★★★
COCO	Large	80/91	Medium	★★★★☆
Cityscapes	Medium	30	High	★★★★★
ADE20K	Medium	150	Mixed	★★★★☆
LVIS	Large	1203	Medium	★★★★☆

Thank You

For Your Attention